

Helm Chart Guide

ال Values.Yaml

هو ملف الإعدادات Configuration يتابعه ال Helm Chart

لوعندنا Template مكتوب فيها

replicas: {{.Values.app.replicas}}

هنا ال Template محتاج يعرف عدد ال replicas

فدعنا نأخذ ال Values.Yaml

ويلاحظي `app` يقول حول `2` ال

replicas: 2

بالتالي في ال Yaml النهائي ستبقى replicas: 2

يبقى ال Values.Yaml بيوفر ال Templates



۱۰۸ | ج ۳ | خ ۱

17

117 **सहस्र**

17 " " : 5175

employment - 11

๕๖/๑๗

Page No. 11

நிர்வாக குழு

15-11-2019

1. Introduction : 10

ਵਰਗਤ ਕਰੋ

1. Micrograph

Registrierung

Helm Chart Guide

File Values.Yaml #

db ← db ال Configuration
name: account ← DB
↑ database ودا اسمي

existingSecret: " " في سطر

Helm معناها ← هنا يومه Secret تم انشاؤه مسبقاً خارج Helm
Kubectl -n vProfile Create Secret بمعنى انا عملت
يمكن احط اسم ال Secret في مكانها

existingSecret: db01-credentials بدل ما تكون فاصلة

ساعتها ال Secret موجودة ومش هيكمل جديد
لكن في الطريقة الثانية ان Jenkins هو اللي
بيمرر القيمة دي وقت ال deploy

انا هعمل Secret ونسبب القيمه خاصية

db:
existingSecret: " "
db01-credentials

Kubectl -n vProfile create generic db-credentials
--from-literal=MYSQL_ROOT_PASSWORD=

ال Kubectl هو اللي عمل ال K8S Secret
ووقت ال Deploy في سطر في Jenkins

--set db.existingSecret=db01-credentials
ساعتها يضيف اسم ال Secret

Helm Chart Guide

File ال Values.yaml

يبدأ ال `existingSecret` فإني

وعملت ال `Secret` ووقت ال `deploy` بـ `Kubectl`

وبعد كذا كتبه امر في `Jenkins` يحل القيمة بتاعها

يبدأ كذا بقية `existingSecret=dbat-Credentials`

وال `Secret` كذا اتعملت واسمها `db-Credentials`

db

`existingSecret: ""`

`rootPassword: ""`

هنا طريقة مختلفة
وفعليه مش بنستخدمها اللبس لو عيت استخدمها
اقدر استخدمها وهي

بدا ما استخدم `Kubectl` بـ `Helm`

هو اللبس يعمل ال `Secret`

وال `Password` يتكرر من خلال ال `Pipeline`

Helm Chart Guide

File Values.yaml

```
Storage
  className: gp3
  size: 8Gi
```

Storage

الجزء دافا لن يتخزين MySQL

Class Name

دا اسم Storage Class لـ Kubernetes

في AWS بنستخدم نوع gp3 في انشاء EBS Volume

General Purpose = gp3

أي هو ال gp3

هو نوع من انواع ال EBS Volume

EBS
gp3
gp2
io2

كل واحد منهم له ضمانات بالنسبة لمشروعنا ال gp3

يعني SSD مخمض للاستخدام العامة

وهو الجيل الثالث من General Purpose SSD

ملاحظة مهمة ال Volume يقدر يخزن 20 GiB

طبيه قد أي ال Volume يقدر يتعامل مع عمليات القراءة

والكتابة ودا يقاس بـ Throughput ، IOPS

ال IOPS Input/Output Operation per Second

ببساطة عدد العمليات القراءة والكتابة اللي يقدر ال Volume

يفعلها فالتانية مثال عنك DB بتعمل

كل عملية من دول تعتبر 1/0 operation

لو عندي 3000 IOPS

ممكن كذا ال Volume يقدر يتعامل مع

3000 عملية 1/0 في الثانية

Read
Write
Read
Write

Helm Chart Guide

File | Values.yaml #

ThroughPut ← طيف أي هو
الـ IOPS بيوتيم بعد العمليات
الـ ThroughPut كمية البيانات التي يتم نقلها في الثانية
و يتم قياس ← MiB/s

Helm Chart Guide

File - Values.yaml

`ingress
enabled: true`

تفعيل ال Ingress

فعل ال Ingress

يعني ال Chart يعمل Kubernetes Ingress Res
هو هنا بيتمل وبقا ال Ingress والانشاء في ملف `ingres.yaml`

تحديد أي Ingress Controller هو المسؤول عن تنفيذ

`ClassName: alb`

ال Ingress ده

دا معناه ان ال Ingress هيتعامل مع ال AWS ALB

بحدد هل ال ALB يكون متاح من الانترنت

`Scheme: internet-facing`

يعني

ال ALB يكون متاح الوصول ليه من الانترنت

احد ال URL Path اللي يستخدم ال ALB

لعمل Health Check على ال backend

يعني ال ALB بيحس بياكد ان ال APP شغال

ايضا Request عالي Login/

يعني ال backend شغال

يحدد كمات كل ال Values أو ال Metadata

لا app, MQ, Cache, DB

تحديد replicas, limit